Recitation 2 Solutions

If-Else Statements

Problem 1

Part I

Suppose a course assigns a final status to each student based on both the exam score and homework completion rate. Write an `if-else` statement that assigns a value to a variable called `status` according to the following rules:

- `Excellent Standing` if the `exam_score` is greater than or equal to `90` and the `homework_rate` is greater than or equal to `0.9`
- `Good Standing` if the `exam_score` is greater than or equal to `80` and the `homework_rate` is greater than or equal to `0.80`
- `Conditional Pass` if the `exam_score` is greater than or equal to `70` and the `homework_rate` is greater than or equal to `0.95`
- `Fail` otherwise.

Test your code with `exam_score = 84` and `homework_rate = 0.92`. Make sure your code prints the value of `status`.

```
exam_score <- 84

homework_rate <- 0.92
```

```
if (exam_score >= 90 & homework_rate >= 0.90) {

  status <- "Excellent Standing"

} else if (exam_score >= 80 & homework_rate >= 0.80) {

  status <- "Good Standing"

} else if (exam_score >= 70 | homework_rate >= 0.95) {

  status <- "Conditional Pass"

} else {

  status <- "Fail"
}

print(status)
```

```
## [1] "Good Standing"
```

Part II

Now consider:

```
exam_scores <- c(94, 82, 76, 68, 88, 91, 73, 59)

homework_rates <- c(0.95, 0.84, 0.98, 0.90, 0.72, 0.88, 0.79, 1.00)
```

Use `ifelse()` to create a logical vector called `eligible` that is `TRUE` if a student:

- has an exam score of at least 75, **and**
- has completed at least 80% of the homework.

Otherwise, the value should be `FALSE`.

```
eligible <- ifelse(exam_scores >= 75 & homework_rates >= 0.80,

                   TRUE, FALSE)

print(eligible)
```

```
## [1]  TRUE  TRUE  TRUE FALSE FALSE  TRUE FALSE FALSE
```

For Loops

Problem 2

Consider the following matrix of quiz scores (the rows represent students and the columns represent quizzes):

```
scores <- matrix(c(82, 76, 91, 88,

                   67, 85, 79, 92,

                   95, 89, 84, 90,

                   73, 68, 81, 77),

                 nrow = 4,

                 byrow = TRUE)
```

Part I

Create a matrix called `adjusted_scores` with the same dimensions as `scores`. Using `nested for loops`, examine every element of `scores` and apply the following rules:

- if the score is below 70, add 5 points;
- if the score is between 70 and 89 inclusive, add 3 points;

- if the score is 90 or higher, leave it unchanged.

Store the results in `adjusted_scores`.

```
adjusted_scores <- matrix(0, nrow = nrow(scores), ncol = ncol(scores))

for (i in 1:nrow(scores)) {

  for (j in 1:ncol(scores)) {

    if (scores[i, j] < 70) {

      adjusted_scores[i, j] <- scores[i, j] + 5

    } else if (scores[i, j] >= 70 & scores[i, j] <= 89) {

      adjusted_scores[i, j] <- scores[i, j] + 3

    } else {

      adjusted_scores[i, j] <- scores[i, j]

    }
  }
}

print(adjusted_scores)
```

```
##      [,1] [,2] [,3] [,4]
## [1,]   85   79   91   91
## [2,]   72   88   82   92
## [3,]   95   92   87   90
## [4,]   76   73   84   80
```

Part II

Create a logical vector called `all_passed`. For each student, determine whether **all** adjusted quiz scores are at least 70.

```
all_passed <- logical(nrow(adjusted_scores))

for (i in 1:nrow(adjusted_scores)) {

  all_passed[i] <- all(adjusted_scores[i, ] >= 70)
}

print(all_passed)
```

```
## [1] TRUE TRUE TRUE TRUE
```

While Loops

Problem 3

Suppose a process starts with:

```
value <- 5
```

and changes according to the following rule:

- if `value` is even, divide it by 2;
- if `value` is odd, multiply it by 3 and add 1.

This is repeated until the value becomes 1.

Part I

Create an empty vector called `sequence_values`. Using a `while loop`:

- store each value in `sequence_values`;
- update `value` using the rule above;
- stop when `value` becomes 1.

Make sure that `1` is also included as the last element of `sequence_values`.

```
sequence_values <- c()

while (value != 1) {

  sequence_values <- c(sequence_values, value)

  if (value %% 2 == 0) {

    value <- value / 2

  } else {

    value <- 3 * value + 1
  }
}
```

```
sequence_values <- c(sequence_values, 1)

print(sequence_values)
```

```
## [1]  5 16  8  4  2  1
```

Part II

Add a variable:

```
steps <- 0
```

Modify the loop so that `steps` counts how many updates were required before the sequence reached 1.

```
value <- 3

sequence_values <- c()

steps <- 0


while (value != 1) {

  sequence_values <- c(sequence_values, value)

  if (value %% 2 == 0) {

    value <- value / 2

  } else {

    value <- 3 * value + 1
  }

  steps <- steps + 1
}

sequence_values <- c(sequence_values, 1)

print(sequence_values)
```

```
## [1]  3 10  5 16  8  4  2  1
```

```
print(steps)
```

```
## [1] 7
```

Functions

Note: You are allowed to use any built-in summary functions such as `mean()` (for average), `median()` (for median), `sum()` (for summation), `abs()` (for absolute value) etc.

Problem 4

Create a function named `function1` with a single argument `x` that counts the number of even and odd numbers in the input list. The function should return a list with two elements: `Even` representing the count of even numbers and `Odd` representing the count of odd numbers.

Test the function with the following input list: [[1, 3.4, 2, -5], [2.2, 16, -3, 7], [“final”, “any”, “can”, “maybe”, “world”]].

```r
list1 <- list(c(1, 3.4, 2, -5), c(2.2, 16, -3, 7), c("final", "any", "can", "maybe", "world"))


function1 <- function(x){

Even <- c(); Odd <- c()

for (i in 1:length(x)){

  if(is.numeric(x[[i]])){

   for (j in 1:length(x[[i]])) {

     if(x[[i]][j] %% 2 == 0){

       Even = c(Even, x[[i]][j])

     } else if (x[[i]][j] %% 2 == 1){

       Odd = c(Odd, x[[i]][j])

     }

   }

  }

}

return(list(Even = length(Even), Odd = length(Odd)))

}

function1(list1)
```

```
## $Even
## [1] 2
##
## $Odd
## [1] 4
```

Problem 5

Write a function `neighbor_matrix(n, m)` that creates and returns an `n x m` numeric matrix filled according to the following rules:

1. The top-left entry (row 1, column 1) should start with the value 1.
2. Each subsequent entry is filled using the following rule:

- The value of a cell is equal to the sum of its immediate top and left neighbors (if they exist).

- If a neighbor does not exist (for example, for the first row or first column), treat that neighbor as 0.

Test your function with `n = 4` and `m = 5`.

```
neighbor_matrix <- function(n, m) {

  mat <- matrix(0, nrow = n, ncol = m)

  for (i in 1:n) {

    for (j in 1:m) {

      if (i == 1 & j == 1) {

        mat[i, j] <- 1

      } else {

        top <- ifelse(i > 1, mat[i - 1, j], 0)

        left <- ifelse(j > 1, mat[i, j - 1], 0)

        mat[i, j] <- top + left
      }
    }
  }

  return(mat)
}


print(neighbor_matrix(4, 5))
```

```
##      [,1] [,2] [,3] [,4] [,5]
## [1,]    1    1    1    1    1
## [2,]    1    2    3    4    5
## [3,]    1    3    6   10   15
## [4,]    1    4   10   20   35
```

Additional Practice Problems on Functions

Problem 6

Write a function `largest_odd(x)` that takes a numeric vector `x` and returns the largest odd number in the vector. If there are no odd numbers, return `NULL`.

Test your function with the following vector: [10, 15, 11, 4, 3, 8, 17, 34, 4, 11, 12, 17, 21, 1, 31, 36]

```
x <- c(10, 15, 11, 4, 3, 8, 17, 34, 4, 11, 12, 17, 21, 1, 31, 36)
```

```r
largest_odd <- function(x) {

  odd_numbers <- c()

  for (i in 1:length(x)) {

    if (x[i] %% 2 != 0) {

      odd_numbers <- c(odd_numbers, x[i])

    }
  }

  if (length(odd_numbers) == 0) {

    return(NULL)

  }

  largest <- odd_numbers[1]

  for (i in 1:length(odd_numbers)) {

    if (odd_numbers[i] > largest) {

      largest <- odd_numbers[i]

    }
  }

  return(largest)
}

largest_odd(x)
```

```
## [1] 31
```

Problem 7

Write a function `first_repeated(x)` that takes a numeric vector `x` and returns the first repeated element. If no element repeats, return `NULL`.

Test your function with the following vector: [10, 15, 11, 4, 3, 8, 17, 34, 4, 11, 12, 17]

```r
x <- c(10, 15, 11, 4, 3, 8, 17, 34, 4, 11, 12, 17)

first_repeated <- function(x) {

  for (i in 2:length(x)) {
```

```
    for (j in 1:(i - 1)) {

      if (x[i] == x[j]) {

        return(x[i])

      }
    }
  }

  return(NULL)
}

first_repeated(x)
```

```
## [1] 4
```

Problem 8

Write a function `generate_custom_sequence(n)` that generates a sequence of numbers where each number is the sum of the previous two numbers, starting with 1 and 2. The function should return the first `n` numbers in the sequence.

Test your function with `n = 10`.

```
n <- 10

generate_custom_sequence <- function(n) {

  sequence <- c(1, 2)

  for (i in 3:n) {

    sequence[i] <- sequence[i - 1] + sequence[i - 2]

  }

  return(sequence)
}

generate_custom_sequence(10)
```

```
##  [1]  1  2  3  5  8 13 21 34 55 89
```

Problem 9

Write a function `longest_run(x)` that takes a numeric vector `x` and returns the longest consecutive sequence of repeated numbers along with its length.

Test your function with `x = c(1, 2, 2, 2, 3, 3, 4, 4, 4, 4, 5, 6)`.

```
x <- c(1, 2, 2, 2, 3, 3, 4, 4, 4, 4, 5, 6)

longest_run <- function(x) {

  current_length <- 1
  longest_length <- 1
  longest_value <- x[1]

  for (i in 2:length(x)) {

    if (x[i] == x[i - 1]) {

      current_length <- current_length + 1

    } else {

      current_length <- 1

    }

    if (current_length > longest_length) {

      longest_length <- current_length
      longest_value <- x[i]

    }
  }

  return(c(longest_value, longest_length))
}

longest_run(x)
```

```
## [1] 4 4
```

Problem 10

Write a function `sum_of_unique(x)` that returns the sum of the elements in a numeric vector `x` that appear only once.

Test your function with `c(2, 3, 4, 3, 5, 2, 7)`.

```
x <- c(2, 3, 4, 3, 5, 2, 7)

sum_of_unique <- function(x) {

  unique_values <- c()

  for (i in 1:length(x)) {
```

```
    count <- 0

    for (j in 1:length(x)) {

      if (x[i] == x[j]) {

        count <- count + 1

      }
    }

    if (count == 1) {

      unique_values <- c(unique_values, x[i])

    }
  }

  return(sum(unique_values))
}

sum_of_unique(x)
```

```
## [1] 16
```

Problem 11

Create a function that takes a list of numeric vectors as input and rotates each vector clockwise by one position [for example (1, 2, 3) becomes (3, 1, 2)]. The function should return a new list with the rotated vectors.

Test your function with the following input list: [(3, 5, 7, 9), (10, 20, 30), (2, 4, 6, 8, 10)]

```
input_list <- list(c(3, 5, 7, 9), c(10, 20, 30), c(2, 4, 6, 8, 10))

rotate_vectors <- function(x) {

  result <- list()

  for (i in 1:length(x)) {

    current_vector <- x[[i]]

    last_value <- current_vector[length(current_vector)]

    remaining_values <- current_vector[-length(current_vector)]

    result[[i]] <- c(last_value, remaining_values)
  }
```

```
  return(result)
}

rotate_vectors(input_list)
```

```
## [[1]]
## [1] 9 3 5 7
##
## [[2]]
## [1] 30 10 20
##
## [[3]]
## [1] 10  2  4  6  8
```

Problem 12

Create a function named `classify_numbers` with a single argument `num_list` that categorizes numbers in the input list into different groups. The function should perform the following steps:

1. Iterate through each element in the input list and check if it is a number. If it is a number, determine whether it is positive, negative, even, or odd.
2. Return a list containing four elements:

- `PositiveEven`: Count of positive even numbers
- `PositiveOdd`: Count of positive odd numbers
- `NegativeEven`: Count of negative even numbers
- `NegativeOdd`: Count of negative odd numbers

Test your function with the following input list:

```
num_list <- list(1, -2, 3, c(4, -3), -5, 6, c("text", "dot", "period"), -8, 9)
```

```
classify_numbers <- function(num_list) {

  PositiveEven <- 0
  PositiveOdd <- 0
  NegativeEven <- 0
  NegativeOdd <- 0

  for (i in 1:length(num_list)) {

    if (is.numeric(num_list[[i]])) {

      current_element <- num_list[[i]]

      for (j in 1:length(current_element)) {

        current_number <- current_element[j]
```

```
        if (current_number > 0 & current_number %% 2 == 0) {

          PositiveEven <- PositiveEven + 1

        } else if (current_number > 0 & current_number %% 2 != 0) {

          PositiveOdd <- PositiveOdd + 1

        } else if (current_number < 0 & current_number %% 2 == 0) {

          NegativeEven <- NegativeEven + 1

        } else if (current_number < 0 & current_number %% 2 != 0) {

          NegativeOdd <- NegativeOdd + 1

        }
      }
    }
  }

  return(list(PositiveEven = PositiveEven, PositiveOdd = PositiveOdd,

              NegativeEven = NegativeEven, NegativeOdd = NegativeOdd))
}

classify_numbers(num_list)
```

```
## $PositiveEven
## [1] 2
##
## $PositiveOdd
## [1] 3
##
## $NegativeEven
## [1] 2
##
## $NegativeOdd
## [1] 2
```

Problem 13

Create a function named `update_sensor_readings` that takes two arguments: a matrix representing sensor readings from multiple machines across several days, and a vector containing specific outlier values to be removed. The function should replace all occurrences of each value in the vector with the average of its immediate neighboring values (up, down, left, right) in the matrix. If the outlier is on the edge or corner of the matrix and doesn't have four neighbors, only use the available neighbors to compute the average.

For instance, test your function with the following matrix and outlier vector:

```r
sensor_matrix <- matrix(c(23.2, 12, 10, 15, 32, 55, 10, 4, 56, 45, 10, 30),

                        nrow=3, byrow=TRUE)

outliers <- c(10, 55)
```

Specifically, you should remove all occurrences of the outliers (10 and 55) by replacing them with the average of their neighboring values.

```r
update_sensor_readings <- function(sensor_matrix, outliers) {

  updated_matrix <- sensor_matrix

  for (i in 1:nrow(sensor_matrix)) {

    for (j in 1:ncol(sensor_matrix)) {

      if (sensor_matrix[i, j] %in% outliers) {

        neighbors <- c()

        # Up
        if (i > 1) {
          neighbors <- c(neighbors, sensor_matrix[i - 1, j])
        }

        # Down
        if (i < nrow(sensor_matrix)) {
          neighbors <- c(neighbors, sensor_matrix[i + 1, j])
        }

        # Left
        if (j > 1) {
          neighbors <- c(neighbors, sensor_matrix[i, j - 1])
        }

        # Right
        if (j < ncol(sensor_matrix)) {
          neighbors <- c(neighbors, sensor_matrix[i, j + 1])
        }

        updated_matrix[i, j] <- mean(neighbors)
      }
    }
  }

  return(updated_matrix)
}

update_sensor_readings(sensor_matrix, outliers)
```

```
##      [,1]  [,2]     [,3] [,4]
## [1,] 23.2 12.00 12.33333   15
```

```
## [2,] 32.0 24.75 19.75000    4
## [3,] 56.0 45.00 28.33333   30
```

Problem 14

Write a function `iterative_trim(x, k)` that repeatedly removes the element farthest from the median until `k` elements remain. Note, after removing each element, you must recalculate the median of the remaining vector before deciding which element to remove next.

Test your function with `x = (1, 40, 2, -3, 10, 15, 7, 5, 5, 0, -4)` and `k = 5`.

```
iterative_trim <- function(x, k) {

  if (k >= length(x)) {

    return(x)
  }

  remaining <- x

  while (length(remaining) > k) {

    med <- median(remaining)

    dist <- abs(remaining - med)

    farthest_idx <- which.max(dist)

    remaining <- remaining[-farthest_idx]
  }

  return(remaining)
}

x = c(1, 40, 2, -3, 10, 15, 7, 5, 5, 0, -4); k = 5

iterative_trim(x, k)
```

```
## [1] 1 2 5 5 0
```

Problem 15

Write a function `count_sign_changes(x)` that counts how many times a numeric vector changes sign (from positive to negative or vice versa). For example, it should return 3 for `x = c(-2, -1, 1, 3, -4, 5)`.

Test your function with `x = c(-10, 1, 3, -5, -4, 2, 3, -1, 6, -9)`.

```r
count_sign_changes <- function(x) {

  count <- 0

  for (i in 1:(length(x) - 1)) {

    if ((x[i] > 0 & x[i + 1] < 0) | (x[i] < 0 & x[i + 1] > 0)) {

      count <- count + 1
    }
  }

  return(count)
}


x <- c(-10, 1, 3, -5, -4, 2, 3, -1, 6, -9)

count_sign_changes(x)
```

```
## [1] 6
```

Problem 16

Write a function `find_plateau(x)` that detects the first flat region in a numeric vector, meaning consecutive equal values of length ≥ 3. The function should return the start and end indices of that plateau, or `NULL` if no plateau exists.

Test your function with `x = (1, 3, 3, 6, 2, 7, 8, 2, 5, 5, 5, 2)`.

```r
find_plateau <- function(x) {

  n <- length(x); start <- NULL; count <- 1

  for (i in 2:n) {

    if (x[i] == x[i - 1]) {

      count <- count + 1

      if (count == 3 & is.null(start)) {

        start <- i - 2
      }

    } else {

      if (!is.null(start) & count >= 3) {

        end <- i - 1
```

```
        return(c(start = start, end = end))

      }

      count <- 1

      start <- NULL
    }
  }


  if (!is.null(start) & count >= 3) {

    end <- n

    return(c(start = start, end = end))
  }

  return(NULL)
}


x <- c(1, 3, 3, 6, 2, 7, 8, 2, 5, 5, 5, 2)

find_plateau(x)
```

```
## start   end
##     9    11
```